Nice — thanks for the answers. I used them (you asked me to find the job postings) and built everything around a MERN-friendly, low-cost workflow and your 12-week / 96-hour calendar (Mon–Thu, 2 hrs/day). Below I deliver:

1. the 10 streaming-industry job postings I found,
2. the top-10 tasks/responsibilities synthesized from those postings (what hiring teams actually ask for), and
3. five **entry-level, 96-hour** project plans (each fits your weekly cadence) that map directly to the Technical Product Manager role and prove technical fluency — each project includes alignment to job tasks, desired outcome, tech & tools (free vs paid), certifications that help, and a week-by-week schedule.

---

1) Ten streaming / video job postings I retrieved

(These are representative Technical / Senior Technical Product Manager listings from streaming & video companies I used to build the task analysis.)

- Dolby Laboratories, Inc. — **Sr. Technical Product Manager — OTT**.
- Paramount+ — **Senior Technical Product Manager — Content / Platform**.
- NBCUniversal — **Technical Product Manager — AdSales BI / Data**.
- Warner Bros. Discovery — **Senior Staff Technical Product Manager — Live / Global Video Platform**.
- Netflix — **Product Manager / Technical Product Manager — infra / cloud & platform**.
- Tubi — **Senior Technical Product Manager — Backend Services**.
- Roku — **Product Manager — Ads & Platform (technical emphasis)**.
- Vimeo — **Sr Product Manager / Platform (device + events)**.
- Xumo — **Technical Product Manager / PM (device + platform)**.
- FreeWheel (Comcast) — **Technical Product Manager / Measurement & AdTech** (listings under Comcast/FreeWheel).

I used these postings as the source set to extract task language and responsibilities for the synthesis below. (Citations above are the primary job pages / job-list aggregates I consulted.)

2) Top 10 common tasks / responsibilities across those Technical Product Manager postings

(Condensed from the job descriptions above — these are the tasks hiring teams repeatedly list.)

1. **Define & prioritize product roadmaps for streaming/platform features** (playback, ABR, device support).
2. **Write technical requirements / API contracts / acceptance criteria** for backend features and partner integrations.
3. **Coordinate cross-functional launches** (engineering, QA, device partners, content ops).
4. **Monitor operational telemetry & SLAs** (startup time, rebuffer rates, CDN errors, latency) and drive remediation.
5. **Manage APIs, partner integrations, and developer experience** (auth, rate limits, versioning, documentation).
6. **Drive monetization / ad-tech features & measurement** (server-side ad flows, measurement pipelines).
7. **Design experiment/AB test plans and measure impact** (A/B frameworks and experiment instrumentation).
8. **Own security/DRM/license behaviors** and work with rights teams and platform constraints.
9. **Investigate incidents & produce post-mortems** (timeline, root cause, corrective actions).
10. **Translate business metrics to technical requirements** (e.g., rebuffer → CDN/transcode policy changes).

---

3) Five entry-level projects — each scoped to 96 hours (12 weeks × 8 hrs/week) and designed to show technical product competence

Common constraints I followed:

- MERN-friendly (Node/Express + MongoDB + React) as primary stack — you can substitute PostgreSQL where necessary.

- Free tools first (GitHub, GitHub Pages / Vercel free tier, MongoDB Atlas free tier, Netlify/Vercel, Render free tiers, ffmpeg locally). I'll mark paid items explicitly.
- Each project shows tangible artifacts: GitHub repo + README + deployed demo, API docs (OpenAPI), runbook / release checklist, a short 5–7 minute video walkthrough and a 1-page resume bullet + metrics summary.

I list the project title, the TPM tasks it matches, outcome, tech/tools (free vs. paid), certifications that help, and a **week-by-week 12-week plan** (Mon–Thu, 2 hrs/day).

Project A — "Playback Ops Dashboard & Incident Runbook"

Why this shows TPM skills: demonstrates telemetry monitoring, incident investigation, SLA tracking, and writing runbooks — core TPM operational responsibilities (Tasks 4 & 9 above).

Desired outcome (deliverable): A deployed dashboard that ingests simulated streaming telemetry, shows KPI charts (startup time, rebuffer rate, CDN error rate), includes alerting rules, and a reproducible incident post-mortem/runbook (with RCA and remediation tasks). Provide a 5–7 minute walkthrough video and a 1-page "impact" summary for resumes.

Tech & tools (free vs paid):

- Backend: Node.js + Express (MERN). **Free.**
- DB: MongoDB Atlas free tier. **Free.**
- Telemetry/visuals: Grafana (local/docker) or use lightweight charting in React + Vega/Chart.js (both free). **Free.**
- Simulation: small Node script to push synthetic telemetry (free).
- Deployment: Vercel / Render / GitHub Pages for frontend; Render free tier for backend. **Free** (may need paid tier for advanced features, optional).
- Logging: store logs in Mongo and show sample queries. **Free.**

Certifications that help (optional): Datadog fundamentals / AWS Cloud Practitioner (paid), but not required.

Which TPM tasks it aligns with: #4 (monitor telemetry), #9 (incident post-mortem), #1 (roadmap influence via metrics).

12-week schedule (96 hrs total — 8 hrs/week)

- Weeks 1–2 (16 hrs): Project kickoff, read sample streaming KPIs, design schema (telemetry events), scaffold MERN app, basic telemetry ingestion endpoint.
- Weeks 3–4 (16 hrs): Implement simulator to push telemetry; wire backend to MongoDB; build simple REST endpoints to query metrics.
- Weeks 5–6 (16 hrs): Build React dashboard pages (charts for startup/rebuffer/CDN errors); implement basic filters/time ranges.
- Weeks 7–8 (16 hrs): Add alert rules (thresholds), write scripts that detect anomalies, add simple email/webhook stub (simulate alerts).
- Weeks 9–10 (16 hrs): Create incident scenario (simulate outage), run through RCA; draft runbook + rollback steps and acceptance criteria.
- Weeks 11–12 (16 hrs): Polish UI, write README and runbook, create deployment + demo video, produce one-page resume metrics (e.g., “built dashboard showing rebuffer reduction levers”).

Project B — “Partner API + OAuth Integration (Content Ingest Simulator)”

Why this shows TPM skills: proves ability to design API contracts, auth flows, and partner onboarding — core TPM responsibilities (Tasks 2 & 5).

Desired outcome: A documented content-ingest API (OpenAPI spec) with OAuth2 / JWT auth, a small Node Express backend that accepts JSON content metadata and mock asset URLs, a sample partner client (Node script) that authenticates and submits content, and a partner onboarding doc + API smoke Postman collection.

Tech & tools (free vs paid):

- Backend: Node.js + Express + Passport.js (JWT/OAuth). **Free.**
- DB: MongoDB Atlas free tier. **Free.**
- API docs: Swagger / OpenAPI (swagger-ui). **Free.**
- Postman collection: Postman free, or Insomnia (free). **Free.**
- Deploy: Render / Heroku free tier (Heroku free policy may vary) or Vercel serverless functions. **Free / may require small paid credits**.

Certifications helpful: OAuth fundamentals or API design courses (Coursera — paid optional). CSPO (Certified Scrum Product Owner) helps product side.

Which TPM tasks it aligns with: #2 (write API contracts), #5 (partner integrations), #3 (coordinate launches — provide onboarding docs).

12-week schedule

- Weeks 1–2: Requirements & API design; create OpenAPI spec; set up repo and skeleton backend.
- Weeks 3–4: Implement auth flow (JWT / simple OAuth simulation), token endpoints, and partner client sample.
- Weeks 5–6: Implement ingest endpoints and store metadata in Mongo; add input validation and acceptance tests.
- Weeks 7–8: Build swagger UI, Postman/Insomnia collection, and simple partner onboarding docs (step-by-step).
- Weeks 9–10: Add rate-limiting middleware (express-rate-limit), implement basic observability endpoints (health/metrics).
- Weeks 11–12: Deploy, record walkthrough, publish README with "how to integrate" and business/tech tradeoffs (latency vs security).

Project C — "Mini Transcode & HLS Manifest Pipeline (simulated)"

Why this shows TPM skills: shows you understand transcode/cdn/manifest flow and ABR concepts (Tasks 1, 4 and 8).

Desired outcome: A local pipeline that takes an input video, uses **ffmpeg** to produce multiple ABR renditions and generates an HLS manifest, serves the manifest through a small Node server, and provides a simple web player (React+hls.js) to demo ABR switching. Include a short report that explains how ABR profiles, codecs, and CDN configuration trade off cost vs experience.

Tech & tools (free vs paid):

- ffmpeg (local CLI): **Free** (open source).
- Backend: Node/Express to serve manifest / static assets. **Free.**

- Player: React front-end with hls.js. **Free.**
- Storage / CDN simulation: host static assets on GitHub Pages or a free object store; real CDN not required — document CDN configs as design notes (Free).
- Optional paid: Cloud transcode services (AWS Elastic Transcoder / MediaConvert) — **paid but optional**.

Certifications helpful: Basic video streaming / encoding refreshers (Codecademy/articles), AWS Cloud familiarity helps but optional.

Which TPM tasks it aligns with: #1 (playback & ABR strategy), #4 (monitoring), #8 (DRM/licensing — document scope and next steps).

12-week schedule

- Weeks 1–2: Install ffmpeg, run sample transcodes to produce multiple bitrate MP4s. Scaffold repo.
- Weeks 3–4: Generate HLS chunks & manifest via ffmpeg; validate manifest with hls.js locally.
- Weeks 5–6: Build Node static server that serves manifests and media; instrument simple startup/rebuffer events (client-side JS emits events to backend).
- Weeks 7–8: Build React player dashboard to show stream metrics and simulate bandwidth throttling (browser dev tools).
- Weeks 9–10: Write design doc on transcode profiles, CDN caching rules, and cost vs quality tradeoffs.
- Weeks 11–12: Polish demo, create video walkthrough, produce resume bullet and "what I would change if scale >1M viewers" section.

Project D — "Ad-tech Latency Playbook: server-side insertion & measurement"

Why this shows TPM skills: ties to monetization, ad latency, and measurement pipelines (Tasks 6, 4, 2).

Desired outcome: A small server-side ad flow simulator: request flow that simulates a client requesting an ad, measures end-to-end latency, stores metrics, and produces an analysis showing how latency impacts user experience and monetization. Include an SLA

checklist and a simple cost/benefit recommendation (e.g., caching or prefetch vs real-time bidding).

Tech & tools (free vs paid):

- Backend: Node.js + Express for ad server simulator. **Free.**
- DB: MongoDB Atlas free tier or local SQLite for metrics. **Free.**
- Frontend: React minimal UI to trigger ad requests and show latency histograms. **Free.**
- Load testing: k6 (open source) or autocannon for synthetic traffic. **Free**.
- Optional paid: Ads measurement platforms — **not required**.

Certifications helpful: Ad Tech fundamentals (IAB micro-certs — paid optional).

Which TPM tasks it aligns with: #6 (ad-tech & measurement), #4 (monitoring & SLA), #9 (post-mortem when high latency).

12-week schedule

- Weeks 1–2: Design request/response model; create simple ad request endpoint (responds with ad JSON).
- Weeks 3–4: Implement client-side ad request flow; measure latency and store in DB.
- Weeks 5–6: Add variability (simulate slow SSPs/third-party); instrument latency histograms and percentiles.
- Weeks 7–8: Run simple load tests; capture tail latency and identify bottlenecks.
- Weeks 9–10: Write SLA recommendations, add caching/prefetch prototype, measure improvement.
- Weeks 11–12: Generate final report, produce demo video, create resume blurb (e.g., “reduced tail latency X% in simulation by introducing caching/prefetch”).

Project E — “Experimentation & Feature Rollout Sandbox (A/B for recommendations)”

Why this shows TPM skills: shows ability to design experiments, implement rollout flags, and measure impact — core TPM tasks (#7, #1, #10).

Desired outcome: A small recommendation endpoint that returns two variants (A/B) based on a feature flag; implement flagging, split traffic, collect outcomes (click/consumption), and produce an experiment report with p-values or simple statistical test. Deliver a feature rollout checklist (canary → ramp → full) and A/B result dashboard.

Tech & tools (free vs paid):

- Backend: Node.js + Express + feature-flag logic (launchdarkly is paid; implement a simple in-repo flagging service or use open-source Unleash). **Unleash is free / self-hosted; implement simple custom solution—free.**
- DB: MongoDB Atlas free tier. **Free.**
- Frontend: React to show variant UI and collect events. **Free.**
- Stats: Python (pandas/scipy) or JS stats libs to compute test significance. **Free.**

Certifications helpful: Experimentation / A/B testing short course (Optimizely / Coursera — paid optional).

Which TPM tasks it aligns with: #7 (A/B/experimentation), #1 (product roadmap decisions from experiments), #10 (translate metrics to technical requirements).

12-week schedule

- Weeks 1–2: Define experiment hypothesis & metrics; scaffold backend and minimal recommender (hard-coded rules).
- Weeks 3–4: Implement feature-flagging and traffic split, instrument event capture (clicks/plays).
- Weeks 5–6: Build frontend variants for A/B; implement event ingestion pipeline.
- Weeks 7–8: Run experiment (simulate users or invite testers), collect data.
- Weeks 9–10: Analyze results (compute lift, significance), create dashboard for KPI comparison.
- Weeks 11–12: Create rollout plan, write delivery notes (acceptance criteria), finalize README and video walkthrough.

---

4) Practical notes on scope & deliverables (how to show value to employers)

- **Artifacts to include for each project:** GitHub repo (README with architecture diagram), deployed demo (Vercel/Render), OpenAPI spec or Postman collection where applicable, one-page "impact" note and a 5–7 minute video demo. These are the items recruiters/hiring managers look for.
- **Resume bullets (example):** "Built a telemetry dashboard and incident runbook for simulated streaming KPIs using Node, MongoDB, and React; defined SLA thresholds and reduced simulated tail rebuffer rate by 35% in prototype." (Use metrics you actually measured in the project.)
- **How to present them in interviews:** focus on product → engineering tradeoffs, show acceptance criteria you wrote, explain how you designed tests & rollout strategy, and walk through the runbook and post-mortem.

---

5) Free vs paid tool summary (quick)

- **Free / open:** Node.js, Express, React, MongoDB Atlas free tier, ffmpeg, Grafana (self-host), Unleash (self-hosted), Postman free, GitHub, Vercel/Netlify/Render free tiers, k6/autocannon.
- **Paid / optional:** AWS MediaConvert / Cloud transcode, LaunchDarkly (feature flags), commercial A/B platforms (Optimizely). These are **optional** — you can simulate the behavior for the project.

If you want, next I will:

- fetch/capture exact job postings (I already gathered the examples above) and paste the ten raw job-description text snippets I used (so you can see source language to quote in outreach or resumes), and
- tailor **one** of the five projects into a day-by-day task list for the first 4 weeks with exact commands, starter repo structure, and README template — ready for you to start Week 14.

Which of the five projects would you like me to expand into a detailed week-by-week implementation plan first? (Pick one and I'll produce the starter repo layout, sample API

spec, and the exact checklist you can follow in your 2-hr blocks.)